

AutoChain Emma+

Manuel fonctionnel, technique et opérationnel

Gestion de flotte, identité wallet, certification Ethereum, documents, alertes, carburant, maintenance et vente à double validation.

Version	1.0
Date	Juillet 2026
Environnement	Laravel 13 · PostgreSQL · Vue 3 · Hardhat / Sepolia
Contact	contact@autochain-emma.com · +242 06 878 18 44

Sommaire

- 1 Objet et périmètre
- 2 Travaux réalisés et architecture
- 3 Installation et exploitation
- 4 Rôles et autorisations
- 5 Authentification, MFA et wallet
- 6 Gestion des véhicules
- 7 Kilométrage et carburant
- 8 Maintenance certifiée
- 9 Documents, IPFS et alertes
- 10 Vente à double validation
- 11 Cycle transactionnel et timeline
- 12 Sécurité
- 13 Hardhat et Sepolia
- 14 Tests, CI et livraison
- 15 Dépannage
- 16 Checklist d'exploitation

1. Objet et périmètre

AutoChain Emma+ est une application de gestion de flotte qui sépare les données administratives des preuves techniques. PostgreSQL conserve les données métier et Ethereum conserve des empreintes, états et événements vérifiables. Le présent manuel décrit l'utilisation quotidienne, les contrôles de sécurité, l'exploitation locale et le passage au réseau de test Sepolia.

- Périmètre métier : véhicules, affectation, kilométrage, maintenance, documents, carburant, alertes et ventes.
- Périmètre Web3 : identité wallet, rôles contractuels, receipts, événements et réconciliation.
- Principe central : aucune donnée n'est dite certifiée avant la confirmation du receipt et de l'événement attendu.

2. Travaux réalisés et architecture

2.1 Socle applicatif

- Laravel 13 avec PostgreSQL, Eloquent, policies, middleware, notifications, scheduler et jobs idempotents.
- Vue 3, Inertia.js, Tailwind CSS, interface responsive, thèmes clair/sombre et branding administrateur global.
- Routes véhicule sécurisées par UUID et archivage par soft delete.
- Matrice d'autorisations centralisée pour les cinq rôles métier.

2.2 Socle blockchain

- Contrat VehicleRegistry refondu : admin, garages certifiés, chauffeur affecté et propriétaire.
- Kilométrage strictement croissant, statuts critiques, maintenance hashée et transfert en deux transactions.
- Suppression du transfert direct qui contournait la double validation.
- Événements complets indexés dans la timeline ; aucune PII n'est stockée on-chain.

2.3 Couche de confiance

Le navigateur envoie les transactions via MetaMask. Laravel ne détient aucune clé privée. Après soumission du hash, le serveur contrôle le chain ID, l'adresse du contrat, le wallet émetteur, le calldata exact, le statut du receipt et la présence de l'événement attendu.

3. Installation et exploitation

3.1 Prérequis

- PHP 8.3 avec pdo_pgsql, bcmath et gd ; Composer ; Node.js ; PostgreSQL ; MetaMask.
- Pour le local : Hardhat sur 127.0.0.1:8545, chain ID 31337.
- Pour les traitements asynchrones : queue worker et scheduler actifs.

3.2 Services à lancer

Service	Commande
Laravel	php artisan serve
Frontend	npm run dev

Service	Commande
Queue	<code>php artisan queue:work</code>
Scheduler	<code>php artisan schedule:work</code>
Hardhat	<code>cd blockchain && npm run node</code>
Déploiement local	<code>cd blockchain && npm run deploy:localhost</code>

4. Rôles et autorisations

Rôle	Responsabilités principales
Super Admin	Comptes, véhicule signé, affectation, statut, garage certifié, proposition et annulation de vente.
Gestionnaire	Administration de flotte, documents, carburant, alertes et suivi hors opérations contractuelles admin.
Chauffeur	Accès au véhicule affecté, kilométrage signé et plein carburant.
Garagiste agréé	Maintenance signée uniquement après certification on-chain par l'admin.
Auditeur / Acheteur	Lecture des preuves publiques et acceptation de la vente qui lui est destinée.

IMPORTANT — Le wallet ne remplace jamais le compte Laravel. Les deux identités et le rôle doivent correspondre.

5. Authentification, MFA et wallet

5.1 Connexion applicative

- Les comptes inactifs sont bloqués globalement.
- Le MFA par e-mail utilise un code à six chiffres, une expiration et une limitation des essais.
- Les sessions sont régénérées après authentification et invalidées à la déconnexion.

5.2 Liaison MetaMask

- Ouvrir le menu utilisateur puis Wallet MetaMask.
- Demander un challenge unique et signer le message sans transaction ni gas.
- Laravel récupère cryptographiquement l'adresse, contrôle l'unicité et marque le wallet vérifié.
- Avant chaque transaction, l'application contrôle le réseau et le compte MetaMask sélectionné.

IMPORTANT — Ne jamais transmettre une phrase de récupération ou une clé privée. Les comptes Hardhat sont publics et exclusivement locaux.

6. Gestion des véhicules

6.1 Création

- Seul le Super Admin crée un véhicule on-chain.
- Le VIN, l'immatriculation, la marque, le modèle, l'année, le carburant et la photo sont enregistrés hors chaîne.
- Un UUID technique est haché puis envoyé à registerVehicle.
- Après VehicleRegistered, l'état devient confirmed et le kilométrage initial 0 est certifié.

6.2 Affectation et statut

- Le chauffeur doit disposer d'un wallet vérifié.
- assignDriver est signé par l'administrateur du contrat.
- Les changements critiques utilisent updateStatus et sont visibles dans la timeline.
- L'archivage Laravel est un soft delete et ne détruit jamais les preuves blockchain.

7. Kilométrage et carburant

7.1 Kilométrage

- Le véhicule doit déjà être enregistré on-chain.
- La nouvelle valeur doit être strictement supérieure à la valeur certifiée.
- Sont autorisés : admin, chauffeur affecté et garage certifié selon le contrat.
- La base ne change la valeur certifiée qu'après MileageUpdated confirmé.

7.2 Carburant

- Le chauffeur ne peut saisir un plein que pour son véhicule affecté.
- Le compteur du plein ne peut pas régresser par rapport aux relevés précédents.
- La consommation moyenne utilise la distance entre pleins ordonnés et les litres consommés.

8. Maintenance certifiée

- Le Super Admin certifie ou révoque le wallet du garage avec setGarageCertification.
- Le garage utilise exactement le dernier kilométrage certifié du véhicule.
- Laravel construit un JSON canonique contenant identifiants, type, date, kilométrage, détails, pièces et garage.
- Le SHA-256 résultant est identique en base et dans recordMaintenance.
- La maintenance devient certifiée uniquement après MaintenanceRecorded.

9. Documents, IPFS et alertes

9.1 Documents

- Types autorisés : PDF, JPEG, PNG et WebP, avec taille maximale contrôlée.
- Le SHA-256 est recalculé avant téléchargement ; un fichier altéré est bloqué.
- Chaque téléchargement ou vérification est inscrit dans le journal d'accès.
- Les documents privés sont limités aux rôles de gestion.

9.2 IPFS

- Un document n'est public qu'après réception d'un CID valide.
- Le service appelle explicitement le pinning avant d'enregistrer `is_public=true`.
- Le CID et le lien gateway sont exposés dans l'interface.

9.3 Alertes

- Contrôle technique, assurance, entretien par date et seuil kilométrique.
- Une empreinte unique évite les doublons, notamment les alertes de vidange.
- Les e-mails sont envoyés par la queue aux responsables actifs ayant activé les notifications.

10. Vente à double validation

- Le Super Admin choisit un véhicule enregistré et un acheteur au wallet vérifié.
- `proposeTransfer` produit `TransferProposed` ; la vente passe `admin_signed` seulement après confirmation.
- Seul le compte Laravel acheteur et son wallet exact peuvent appeler `acceptTransfer`.
- Après `TransferAccepted`, la vente devient `completed`, le véhicule sold et le chauffeur est retiré.
- Une annulation on-chain utilise `cancelTransfer` et laisse sa preuve transactionnelle.

11. Cycle transactionnel et timeline

Etat	Signification	Effet métier
pending	Demande préparée	Aucune donnée certifiée modifiée
submitted	Hash reçu, receipt attendu	Réconciliation automatique ou manuelle
confirmed	Receipt réussi + événement attendu	État certifié appliqué atomiquement
failed	Revert ou contrôle invalide	Preuve refusée, nouvel essai traçable

Les logs du contrat sont indexés par hash de transaction et index de log. La timeline fusionne ces événements confirmés avec les informations administratives, les documents et les pleins, sans présenter ces derniers comme preuves blockchain.

12. Sécurité

- Politiques et scopes empêchent les accès inter-rôles et filtrent les tableaux de bord.
- Le wallet acheteur, le chauffeur affecté et le garage certifié sont vérifiés avant transaction.
- Le calldata serveur doit correspondre exactement à la fonction et aux arguments préparés.
- Les clés privées ont été retirées de l'environnement Laravel.
- Les fichiers sont stockés hors du répertoire public sauf médias explicitement publiés.
- Les comptes inactifs, tentatives MFA et téléchargements sont contrôlés.

IMPORTANT — Toute ancienne clé privée placée dans un environnement applicatif doit être considérée compromise et remplacée.

13. Hardhat et Sepolia

13.1 Validation locale

- Déployer VehicleRegistry sur Hardhat et régénérer deployment.json.
- Lier des wallets distincts pour Admin, Garage, Chauffeur et Acheteur.
- Exécuter : certification garage, véhicule, affectation, kilométrage, maintenance, proposition et acceptation.

13.2 Sepolia

- Utiliser un nouveau compte de test privé, jamais une clé Hardhat.
- Configurer un RPC Sepolia fiable et approvisionner chaque signataire via un faucet.
- Déployer avec chain ID 11155111 puis relier les wallets des comptes Laravel.
- Rejouer tous les scénarios et conserver les liens d'explorateur.

13.3 Créer le compte AutoChain

- Sur la page de connexion, cliquer sur « Pas de compte ? Créez-en un ».
- Saisir nom, adresse e-mail, mot de passe et confirmation, puis valider l'adresse e-mail si demandé.
- Tout nouveau compte reçoit le rôle Auditeur / Acheteur. Le Super Admin doit attribuer Chauffeur, Garagiste agréé ou Gestionnaire lorsque le scénario l'exige.
- Le changement de rôle applicatif ne certifie pas automatiquement un garage sur Ethereum.

13.4 Créer un wallet MetaMask de test

- Installer uniquement l'extension ou l'application officielle depuis <https://metamask.io/download/>.
- Choisir « Créer un nouveau wallet », définir un mot de passe local et noter la phrase secrète hors ligne.
- Créer de préférence un compte séparé pour les essais Sepolia ; ne jamais importer une clé Hardhat.
- La phrase secrète et la clé privée ne doivent être saisies ni dans AutoChain, ni dans un faucet, ni envoyées par messagerie.

13.5 Afficher Sepolia

- Dans MetaMask, ouvrir Réseaux puis activer « Afficher les réseaux de test ».
- Sélectionner « Sepolia » et vérifier le chain ID 11155111.
- Si une configuration manuelle est indispensable : symbole ETH et explorateur <https://sepolia.etherscan.io/> ; utiliser le RPC fourni par l'hébergeur ou le prestataire retenu.

13.6 Obtenir des Sepolia ETH

- Copier uniquement l'adresse publique du wallet actuellement sélectionné.
- Essayer un fournisseur reconnu : <https://faucets.chain.link/sepolia>, <https://www.alchemy.com/faucets/ethereum-sepolia> ou <https://cloud.google.com/application/web3/faucet/ethereum/sepolia>.
- Coller l'adresse publique, effectuer les contrôles anti-abus demandés puis réclamer les jetons.
- Les conditions et limites varient selon le faucet ; essayer un autre fournisseur en cas d'inéligibilité.
- Vérifier la réception dans MetaMask et sur <https://sepolia.etherscan.io/> en recherchant l'adresse publique.

IMPORTANT — Les Sepolia ETH sont gratuits et n'ont aucune valeur monétaire. Toute personne qui propose de les vendre ou demande la phrase secrète tente probablement une fraude.

13.7 Lier le wallet et obtenir les autorisations

- Dans AutoChain : menu utilisateur → Wallet MetaMask → Lier MetaMask.
- Sélectionner Sepolia et le bon compte, puis signer le challenge gratuit ; cette signature ne consomme pas de gas.
- Pour un Chauffeur : le Super Admin doit ensuite l'affecter au véhicule.
- Pour un Garagiste : le Super Admin doit attribuer le rôle puis certifier son wallet on-chain.
- Pour un Acheteur : le Super Admin doit désigner exactement ce compte dans la proposition de vente.
- Chaque transaction métier consomme une petite quantité de Sepolia ETH sur le wallet qui signe.

Un testeur externe peut utiliser son propre wallet Sepolia et payer son gas, mais il doit aussi disposer d'un compte AutoChain actif avec le bon rôle. Un garage doit être certifié, un chauffeur affecté et un acheteur explicitement désigné.

14. Tests, CI et livraison

- Tests Laravel : comptes inactifs, MFA, rôles, documents, carburant, alertes, transactions et réconciliation.
- Tests Solidity : 25 scénarios couvrant permissions, statuts, kilométrage, maintenance et vente.
- Test d'intégration Laravel vers Hardhat : chain ID, bytecode du contrat et administrateur.
- Build Vite et tests PostgreSQL exécutés dans la CI.
- Le déploiement Sepolia est déclenché seulement après une CI réussie et l'approbation de l'environnement.

15. Dépannage

Symptôme	Cause probable	Action
Mauvais compte MetaMask	Compte sélectionné différent du wallet requis	Changer de compte, recharger puis signer.
Réseau incorrect	Chain ID différent	Sélectionner Hardhat 31337 ou Sepolia 11155111.
Réservé à l'administrateur	Transaction signée par un autre wallet	Utiliser le wallet déployeur du contrat.
Maintenance refusée	Garage non certifié ou km incohérent	Certifier le garage et reprendre le dernier km confirmé.
403	Le rôle ne possède pas la permission	Vérifier le compte connecté et la matrice des rôles.
Transaction submitted	Receipt pas encore disponible	Attendre le worker ou lancer blockchain:reconcile.
Document 409	SHA-256 différent	Restaurer l'original et auditer le stockage.
Logo absent	Aucun logo Super Admin configuré	Profil Admin → téléverser le logo de l'entreprise.

16. Checklist d'exploitation

Quotidien

- Vérifier les alertes critiques et les transactions submitted.
- Contrôler le queue worker et le scheduler.
- Examiner les échecs de notification et les journaux d'accès documentaire.

Avant une démonstration

- Vérifier le RPC, le chain ID, le contrat et le solde de chaque wallet.
- Confirmer les quatre liaisons wallet et la certification du garage.
- Préparer un véhicule neuf pour ne pas réutiliser un identifiant on-chain.
- Tester la création, l'affectation, le kilométrage, la maintenance et la vente.

Sauvegarde et continuité

- Sauvegarder PostgreSQL et les documents locaux.
- Conserver deployment.json et les hashes de transaction.
- Ne jamais sauvegarder les secrets dans le dépôt Git.
- Tester périodiquement la restauration et la commande blockchain:reconcile.

Contact et assistance

E-mail : contact@autochain-emma.com

Téléphone : +242 06 878 18 44

Application : AutoChain Emma+